

FACULTY WORKLOAD AND TIMETABLE MANAGEMENT SYSTEM

Course: Database Management Systems (DBMS)

Student Name: Lingampally Mohith

Roll Number: 25WU0102137

Faculty Name: Dr. Kiran Mayee Adavala

Academic Year: 2025–2029



TABLE OF CONTENTS

1. Cover Page
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL and DML) with Sample Outputs
10. Queries with Outputs
11. UI Design and Screenshots
12. Implementation Details
13. Testing
14. Conclusion and Future Enhancements
15. References

16. Appendix: GitHub Repository

FACULTY WORKLOAD AND TIMETABLE MANAGEMENT SYSTEM

A Project Report Submitted in Partial Fulfillment of the Requirements for the Course
DATABASE MANAGEMENT SYSTEMS (DBMS)

Submitted By:

Lingampally Mohith

Roll Number: 25WU0102137

Under the Guidance of:

Dr. Kiran Mayee Adavala

Academic Year: 2026–2027

2. ABSTRACT

The Faculty Workload and Timetable Management System is a database-driven application designed to streamline the administration of academic entities, faculty workload distributions, and timetable allocations within an educational institution. Managing academic resources manually or via unstructured spreadsheets often leads to scheduling conflicts, data redundancy, inconsistent workload distribution, and administrative inefficiency.

This project addresses these challenges by implementing a robust relational database model in MySQL, managed via a desktop Graphical User Interface (GUI) developed in Python using Tkinter. The system comprehensively manages eleven core entities: Departments, Programmes, Courses, Sections, Faculty, Workload Norms, Assignments, Rooms, Time Slots, Timetable Entries, and Substitutions.

The relational schema strictly enforces entity and referential integrity through primary and foreign key constraints, minimizing data redundancy and maintaining logical consistency across all records. The system provides complete Create, Read, Update,

and Delete (CRUD) operations alongside search capabilities for administrative staff. By combining a normalized relational backend with a user-friendly frontend interface, the application ensures efficient academic scheduling, accurate workload tracking, and seamless handling of faculty substitutions.

3. INTRODUCTION AND PROBLEM STATEMENT

3.1 Background

In modern higher education institutions, effective planning and allocation of academic resources—including faculty members, classrooms, time slots, courses, and student sections—are critical for smooth operational workflows. Faculty members are bound by specific workload norms based on their designation or role, which specify the maximum teaching hours allowed per period. Concurrently, academic timetables must ensure that classroom capacities align with section strengths and that scheduling conflicts across rooms, time slots, and faculty assignments are systematically avoided.

3.2 Problem Statement

Historically, many departments rely on manual entry or fragmented spreadsheets to schedule courses, assign faculty to sections, and handle temporary faculty substitutions. This traditional approach presents several critical challenges:

1. **High Risk of Scheduling Conflicts:** Double-booking faculty members or classrooms into overlapping time slots is common when managed manually.
2. **Workload Inconsistencies:** Without centralized workload tracking, tracking whether a faculty member exceeds or falls short of their designated teaching hours becomes cumbersome and prone to error.
3. **Data Redundancy and Inconsistency:** Updating details such as course names, section capacities, or faculty department affiliations in spreadsheets often leads to duplicate entries, orphaned records, and conflicting copies of data.
4. **Lack of Centralized Substitution Tracking:** Managing temporary faculty substitutions due to leave or administrative reassignments lacks structured history, making auditability difficult.

3.3 Proposed Solution

The proposed Faculty Workload and Timetable Management System introduces a centralized relational database architecture connected to a Python/Tkinter application. By organizing data into distinct, normalized tables and enforcing foreign key relationships, the system guarantees data integrity, prevents orphaned records, streamlines search and retrieval, and simplifies daily administrative updates through an intuitive user interface.

4. OBJECTIVES AND SCOPE

4.1 Objectives

The main objectives of this project are:

- **Manage Master Academic Data:** Store and manipulate structured records for departments, programmes, courses, sections, rooms, and time slots.
- **Faculty & Workload Tracking:** Maintain detailed faculty records mapped to predefined workload norms defining maximum allowable teaching hours.
- **Manage Course Assignments:** Relate faculty members to specific student sections through dedicated assignment entities.
- **Schedule Timetable Entries:** Allocate teaching assignments to available rooms and time slots systematically.
- **Handle Faculty Substitutions:** Provide record-keeping for reassigning timetable slots to substitute faculty when original faculty members are unavailable.
- **Execute Full CRUD Operations:** Enable end-users to insert, view, modify, and delete records across all operational modules seamlessly.
- **Provide Search Functionality:** Support rapid searching and filtering across administrative modules.
- **Ensure Data Integrity:** Enforce referential integrity, primary keys, and foreign key constraints at the MySQL database engine level.
- **Deliver an Accessible User Interface:** Build a clean Python/Tkinter GUI communicating with MySQL via mysql-connector-python.

4.2 Scope

In-Scope Features:

- Full database schema design and normalization up to 3NF for 11 core tables.
- Relational management of primary and foreign keys across all entities.
- Desktop GUI containing dedicated tabs/modules for Dashboard, Departments, Programmes, Courses, Sections, Faculty, Workload Norms, Assignments, Rooms, Time Slots, Timetable Entries, and Substitutions.
- Real-time CRUD and search operations against a MySQL database.

Out-of-Scope (Future Scope):

- Fully automated algorithmic timetable generation using constraint-satisfaction algorithms.
- Web-based multi-tenant user access control and student-facing portals.
- Automated real-time conflict-blocking triggers for overlap detection during live GUI entry.

5. SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

- Operating System: Windows 10 / 11 (64-bit)
- Programming Language: Python 3.10+
- GUI Framework: Tkinter (standard Python library)
- Database Management System: MySQL Server 8.0+
- Database Connector: mysql-connector-python
- Integrated Development Environment (IDE): Visual Studio Code / PyCharm / IDLE
- Version Control: Git & GitHub

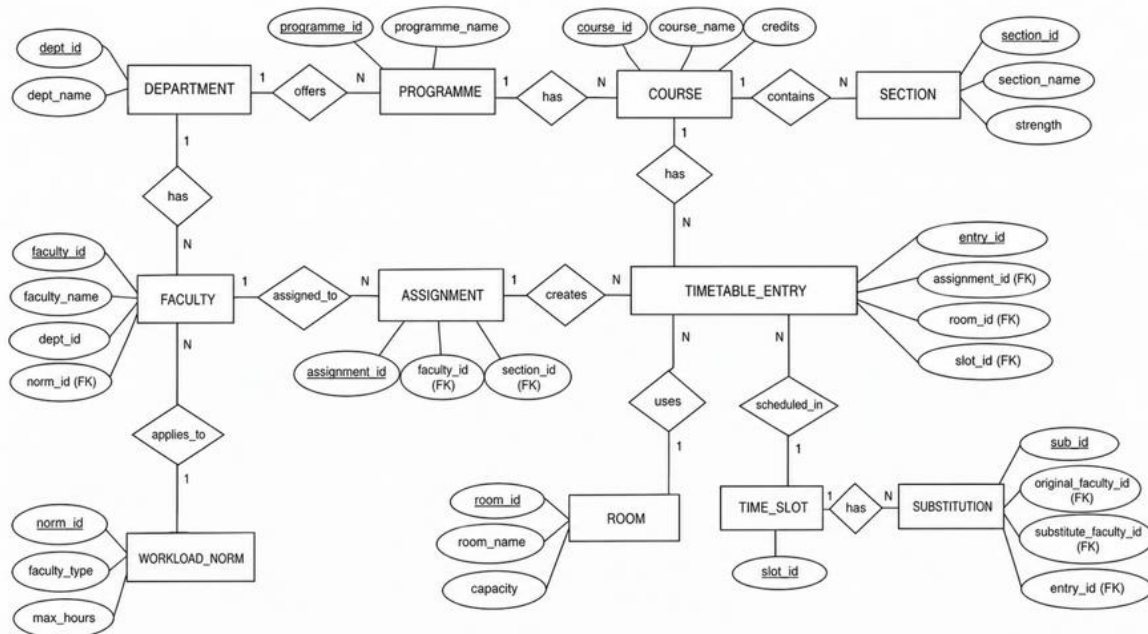
5.2 Hardware Requirements

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (2.0 GHz minimum)
- System Memory (RAM): 4 GB RAM minimum (8 GB recommended)
- Storage: 500 MB free hard drive space (for database files and application source code)
- Display Resolution: 1366 x 768 minimum resolution (1920 x 1080 recommended)

- Input Devices: Standard Keyboard and Mouse / Pointing Device

6. ER DIAGRAM

FIGURE 1: ENTITY-RELATIONSHIP (ER) DIAGRAM



File

Description:

The ER model illustrates the foundational entities of the system and their structural connections. A DEPARTMENT employs multiple FACULTY members, while each FACULTY member operates under a specific WORKLOAD_NORM. An ASSIGNMENT links a FACULTY member to a SECTION. TIMETABLE_ENTRY ties an ASSIGNMENT to a physical ROOM and a TIME_SLOT. Finally, a SUBSTITUTION references an original TIMETABLE_ENTRY, replacing the primary faculty member with a substitute_faculty_id.

7. RELATIONAL SCHEMA AND NORMALIZATION

7.1 Relational Schema

The finalized database design consists strictly of the following 11 tables and their respective primary keys (PK) and foreign keys (FK):

1. DEPARTMENT (dept_id [PK], dept_name)
2. PROGRAMME (programme_id [PK], programme_name)
3. COURSE (course_id [PK], course_name, credits)
4. SECTION (section_id [PK], section_name, strength)
5. WORKLOAD_NORM (norm_id [PK], faculty_type, max_hours)
6. FACULTY (faculty_id [PK], faculty_name, dept_id [FK], norm_id [FK])
7. ASSIGNMENT (assignment_id [PK], faculty_id [FK], section_id [FK])
8. ROOM (room_id [PK], room_name, capacity)
9. TIME_SLOT (slot_id [PK])
10. TIMETABLE_ENTRY (entry_id [PK], assignment_id [FK], room_id [FK], slot_id [FK])
11. SUBSTITUTION (sub_id [PK], original_faculty_id [FK], substitute_faculty_id [FK], entry_id [FK])

7.2 Normalization Analysis

First Normal Form (1NF)

A relation is in 1NF if all attributes contain atomic (indivisible) values, and there are no repeating groups or arrays.

- Compliance: In our schema, all attributes hold singular atomic values (e.g., faculty_name, credits, capacity). Multi-valued attributes like multiple assignments for a single faculty member are represented as separate records in the junction table ASSIGNMENT rather than comma-separated strings inside FACULTY.

Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial dependencies).

- Compliance: Every table in the schema utilizes a simple single-column primary key (e.g., dept_id, faculty_id, assignment_id, entry_id). Because no primary key is composite, partial functional dependencies are mathematically impossible. Thus, all non-key attributes depend entirely on their respective surrogate/primary key.

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key (no non-key attribute determines another non-key attribute).

- Compliance:
 - In FACULTY, faculty_name, dept_id, and norm_id depend directly on faculty_id. The workload details (max_hours) are factored out into WORKLOAD_NORM(norm_id, faculty_type, max_hours), avoiding transitive dependencies like faculty_id -> norm_id -> max_hours.
 - In TIMETABLE_ENTRY, the mapping relates directly entry_id -> (assignment_id, room_id, slot_id) without intermediate dependencies between room and slot attributes.
 - In SUBSTITUTION, all attributes (original_faculty_id, substitute_faculty_id, entry_id) directly depend on sub_id.
 - Hence, the entire relational schema satisfies 3NF requirements.

8. DATA DICTIONARY

Attribute	Data Type	Key Type
Table 1: DEPARTMENT		
dept_id	INT	Primary Key
dept_name	VARCHAR(100)	None

Attribute	Data Type	Key Type
Table 2: PROGRAMME		
programme_id	INT	Primary Key
programme_name	VARCHAR(100)	None
Table 3: COURSE		
course_id	INT	Primary Key
course_name	VARCHAR(100)	None
credits	INT	None
Table 4: SECTION		
section_id	INT	Primary Key
section_name	VARCHAR(50)	None
strength	INT	None
Table 5: WORKLOAD_NORM		
norm_id	INT	Primary Key
faculty_type	VARCHAR(50)	None

Attribute	Data Type	Key Type
-----------	-----------	----------

max_hours	INT	None
-----------	-----	------

Table 6: FACULTY

faculty_id	INT	Primary Key
------------	-----	-------------

faculty_name	VARCHAR(100)	None
--------------	--------------	------

dept_id	INT	Foreign Key
---------	-----	-------------

norm_id	INT	Foreign Key
---------	-----	-------------

Table 7: ASSIGNMENT

assignment_id	INT	Primary Key
---------------	-----	-------------

faculty_id	INT	Foreign Key
------------	-----	-------------

section_id	INT	Foreign Key
------------	-----	-------------

Table 8: ROOM

room_id	INT	Primary Key
---------	-----	-------------

room_name	VARCHAR(50)	None
-----------	-------------	------

capacity	INT	None
----------	-----	------

Attribute	Data Type	Key Type
Table 9: TIME_SLOT		
slot_id	INT	Primary Key
Table 10: TIMETABLE_ENTRY		
entry_id	INT	Primary Key
assignment_id	INT	Foreign Key
room_id	INT	Foreign Key
slot_id	INT	Foreign Key
Table 11: SUBSTITUTION		
sub_id	INT	Primary Key
original_faculty_id	INT	Foreign Key
substitute_faculty_id	INT	Foreign Key
entry_id	INT	Foreign Key

9. SQL COMMANDS USED (DDL AND DML) WITH SAMPLE OUTPUTS

9.1 Data Definition Language (DDL) Commands

```
CREATE DATABASE IF NOT EXISTS workload_timetable_db;
```

```
USE workload_timetable_db;
```

```
CREATE TABLE DEPARTMENT (
```

```
    dept_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    dept_name VARCHAR(100) NOT NULL UNIQUE
```

```
);
```

```
CREATE TABLE PROGRAMME (
```

```
    programme_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    programme_name VARCHAR(100) NOT NULL UNIQUE
```

```
);
```

```
CREATE TABLE COURSE (
```

```
    course_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    course_name VARCHAR(100) NOT NULL,
```

```
    credits INT NOT NULL CHECK (credits > 0)
```

```
);
```

```
CREATE TABLE SECTION (
```

```
    section_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    section_name VARCHAR(50) NOT NULL,  
  
    strength INT NOT NULL CHECK (strength > 0)  
  
);
```

```
CREATE TABLE WORKLOAD_NORM (  
  
    norm_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    faculty_type VARCHAR(50) NOT NULL,  
  
    max_hours INT NOT NULL CHECK (max_hours > 0)  
  
);
```

```
CREATE TABLE FACULTY (  
  
    faculty_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    faculty_name VARCHAR(100) NOT NULL,  
  
    dept_id INT NOT NULL,  
  
    norm_id INT NOT NULL,  
  
    FOREIGN KEY (dept_id) REFERENCES DEPARTMENT(dept_id) ON DELETE  
    CASCADE,  
  
    FOREIGN KEY (norm_id) REFERENCES WORKLOAD_NORM(norm_id) ON DELETE  
    CASCADE  
  
);
```

```
CREATE TABLE ASSIGNMENT (  
  
    assignment_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    faculty_id INT NOT NULL,
```

section_id INT NOT NULL,

FOREIGN KEY (faculty_id) REFERENCES FACULTY(faculty_id) ON DELETE CASCADE,

FOREIGN KEY (section_id) REFERENCES SECTION(section_id) ON DELETE CASCADE

);

CREATE TABLE ROOM (

room_id INT AUTO_INCREMENT PRIMARY KEY,

room_name VARCHAR(50) NOT NULL UNIQUE,

capacity INT NOT NULL CHECK (capacity > 0)

);

CREATE TABLE TIME_SLOT (

slot_id INT AUTO_INCREMENT PRIMARY KEY

);

CREATE TABLE TIMETABLE_ENTRY (

entry_id INT AUTO_INCREMENT PRIMARY KEY,

assignment_id INT NOT NULL,

room_id INT NOT NULL,

slot_id INT NOT NULL,

FOREIGN KEY (assignment_id) REFERENCES ASSIGNMENT(assignment_id) ON DELETE CASCADE,

```
FOREIGN KEY (room_id) REFERENCES ROOM(room_id) ON DELETE CASCADE,  
  
FOREIGN KEY (slot_id) REFERENCES TIME_SLOT(slot_id) ON DELETE CASCADE  
  
);  
  
CREATE TABLE SUBSTITUTION (  
  
    sub_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    original_faculty_id INT NOT NULL,  
  
    substitute_faculty_id INT NOT NULL,  
  
    entry_id INT NOT NULL,  
  
    FOREIGN KEY (original_faculty_id) REFERENCES FACULTY(faculty_id) ON DELETE  
CASCADE,  
  
    FOREIGN KEY (substitute_faculty_id) REFERENCES FACULTY(faculty_id) ON  
DELETE CASCADE,  
  
    FOREIGN KEY (entry_id) REFERENCES TIMETABLE_ENTRY(entry_id) ON DELETE  
CASCADE  
  
);  
  
SHOW TABLES;  
  
DESCRIBE FACULTY;
```

FIGURE 2: MYSQL SHOW TABLES AND DESCRIBE OUTPUT
File

9.2 Data Manipulation Language (DML) Commands

```
INSERT INTO DEPARTMENT (dept_name) VALUES  
  
('Computer Science & Engineering'),
```

```
('Data Science'),  
  
('Electronics & Communication');  
  
INSERT INTO WORKLOAD_NORM (faculty_type, max_hours) VALUES  
  
('Professor', 12),  
  
('Associate Professor', 14),  
  
('Assistant Professor', 16);  
  
INSERT INTO FACULTY (faculty_name, dept_id, norm_id) VALUES  
  
('Dr. Kiran Mayee Adavala', 1, 1),  
  
('Prof. Rajesh Kumar', 1, 3),  
  
('Dr. Anitha Reddy', 2, 2);  
  
UPDATE SECTION SET strength = 65 WHERE section_id = 1;  
  
DELETE FROM SUBSTITUTION WHERE sub_id = 2;  
  
SELECT * FROM FACULTY;
```

FIGURE 3: MYSQL DML OPERATIONS OUTPUT
File

10. QUERIES WITH OUTPUTS

Review 2 Query:

"Show the professor who have work hour more than 18 hrs."

```
mysql> SELECT f.faculty_name, w.faculty_type, w.max_hours
-> FROM FACULTY f
-> JOIN WORKLOAD_NORM w
-> ON f.norm_id = w.norm_id
-> WHERE w.max_hours > 18
-> AND w.faculty_type LIKE '%Professor%';
```

faculty_name	faculty_type	max_hours
Dr. Anil Reddy	Assistant Professor	20.00
Dr. Kiran Kumar	Assistant Professor	20.00

```
2 rows in set (0.01 sec)

mysql> |
```

Query 1: Retrieve Faculty Details with Department Names and Workload Limits

Purpose: Join FACULTY, DEPARTMENT, and WORKLOAD_NORM tables to produce a comprehensive directory of faculty details along with their corresponding workload policy thresholds.

```
SQL:SELECT f.faculty_id, f.faculty_name, d.dept_name, w.faculty_type, w.max_hours
FROM FACULTY f
JOIN DEPARTMENT d ON f.dept_id = d.dept_id
JOIN WORKLOAD_NORM w ON f.norm_id = w.norm_id
ORDER BY f.faculty_id ASC;
```

FIGURE 4: QUERY 1 OUTPUT

File

Explanation: This query connects foreign keys dept_id and norm_id to retrieve user-readable department names and maximum allowable weekly teaching hours instead of raw integer primary key references.

Query 2: Compute Total Number of Teaching Assignments per Faculty Member

Purpose: Aggregate total section assignments per faculty member to help monitor workload distribution.

SQL:

```
SELECT f.faculty_id, f.faculty_name, COUNT(a.assignment_id) AS total_assignments
FROM FACULTY f
LEFT JOIN ASSIGNMENT a ON f.faculty_id = a.faculty_id
GROUP BY f.faculty_id, f.faculty_name
ORDER BY total_assignments DESC;
```

FIGURE 5: QUERY 2 OUTPUT

Explanation: Performs an OUTER JOIN grouped by faculty_id, ensuring faculty members with zero current section assignments are included with a count of 0.

Query 3: Presentation-II Query — Identify Master Timetable Schedules

Purpose: Generate a complete timetable report connecting entries to assignments, sections, rooms, time slots, and assigned faculty members.

SQL:

```sql

```
SELECT te.entry_id, f.faculty_name, sec.section_name, r.room_name, te.slot_id
FROM TIMETABLE_ENTRY te
JOIN ASSIGNMENT a ON te.assignment_id = a.assignment_id
JOIN FACULTY f ON a.faculty_id = f.faculty_id
JOIN SECTION sec ON a.section_id = sec.section_id
JOIN ROOM r ON te.room_id = r.room_id
```

```
JOIN TIME_SLOT ts ON te.slot_id = ts.slot_id
```

```
ORDER BY te.slot_id ASC;
```

FIGURE 6: PRESENTATION-II QUERY OUTPUT

File

Explanation: Combines five distinct tables (TIMETABLE\_ENTRY, ASSIGNMENT, FACULTY, SECTION, ROOM, and TIME\_SLOT) to generate a human-readable master schedule.

### **Query 4: List Faculty Members Assigned to Sections with Capacity Greater than 60**

Purpose: Use nested subqueries to identify faculty taking large sections (e.g., sections with student strength exceeding 60).

```
SQL:SELECT faculty_id, faculty_name FROM FACULTY WHERE faculty_id IN (

 SELECT faculty_id FROM ASSIGNMENT WHERE section_id IN (

 SELECT section_id FROM SECTION WHERE strength > 60

)

);
```

FIGURE 7: QUERY 4 OUTPUT

File

Explanation: Demonstrates nested IN subqueries traversing from SECTION filter criteria through ASSIGNMENT up to FACULTY.

### **Query 5: Active Faculty Substitution Tracking Log**

Purpose: Fetch detailed substitution logs highlighting the original faculty member, substitute faculty member, and corresponding entry details.

```
SQL:SELECT s.sub_id, orig.faculty_name AS original_faculty, sub.faculty_name AS
substitute_faculty, s.entry_id
```

FROM SUBSTITUTION s

JOIN FACULTY orig ON s.original\_faculty\_id = orig.faculty\_id

JOIN FACULTY sub ON s.substitute\_faculty\_id = sub.faculty\_id

ORDER BY s.sub\_id ASC;

FIGURE 8: QUERY 5 OUTPUT

File

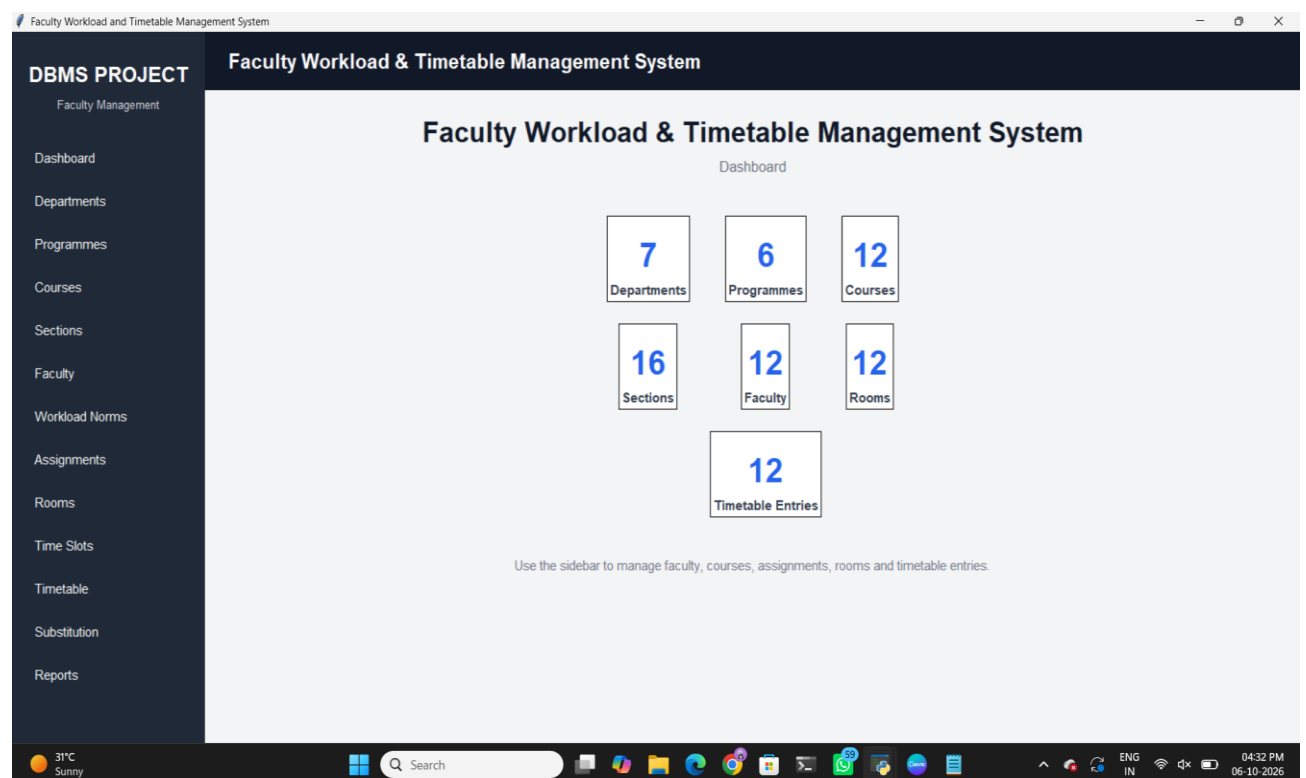
Explanation: Uses a dual join against the FACULTY table using aliases orig and sub to differentiate original faculty from substitute faculty.

## 11. UI DESIGN AND SCREENSHOTS

### 11.1 Dashboard Module

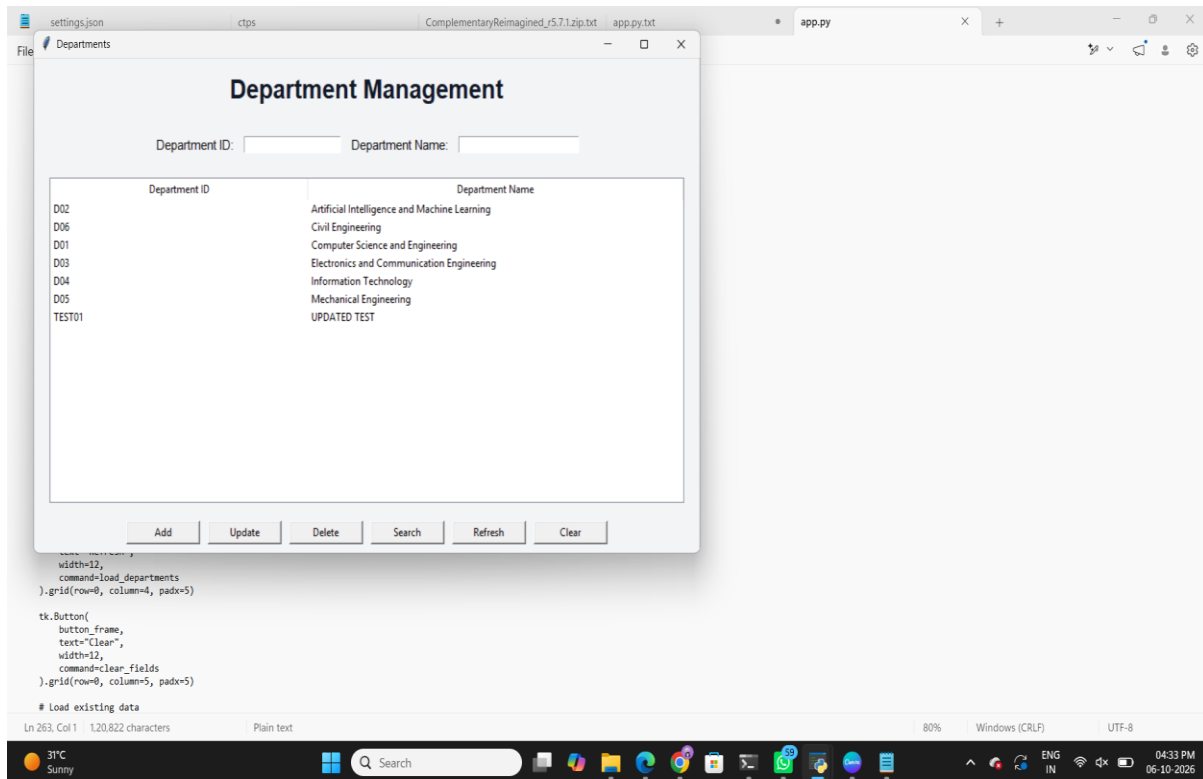
FIGURE 9: DASHBOARD MODULE

File



## 11.2 Departments Module

FIGURE 10: DEPARTMENTS MODULE  
File



## 11.2 Departments Course

Faculty Workload and Timetable Management System

### DBMS PROJECT

#### Faculty Workload & Timetable Management System

Course Management

Course ID:  Course Name:  Credits:

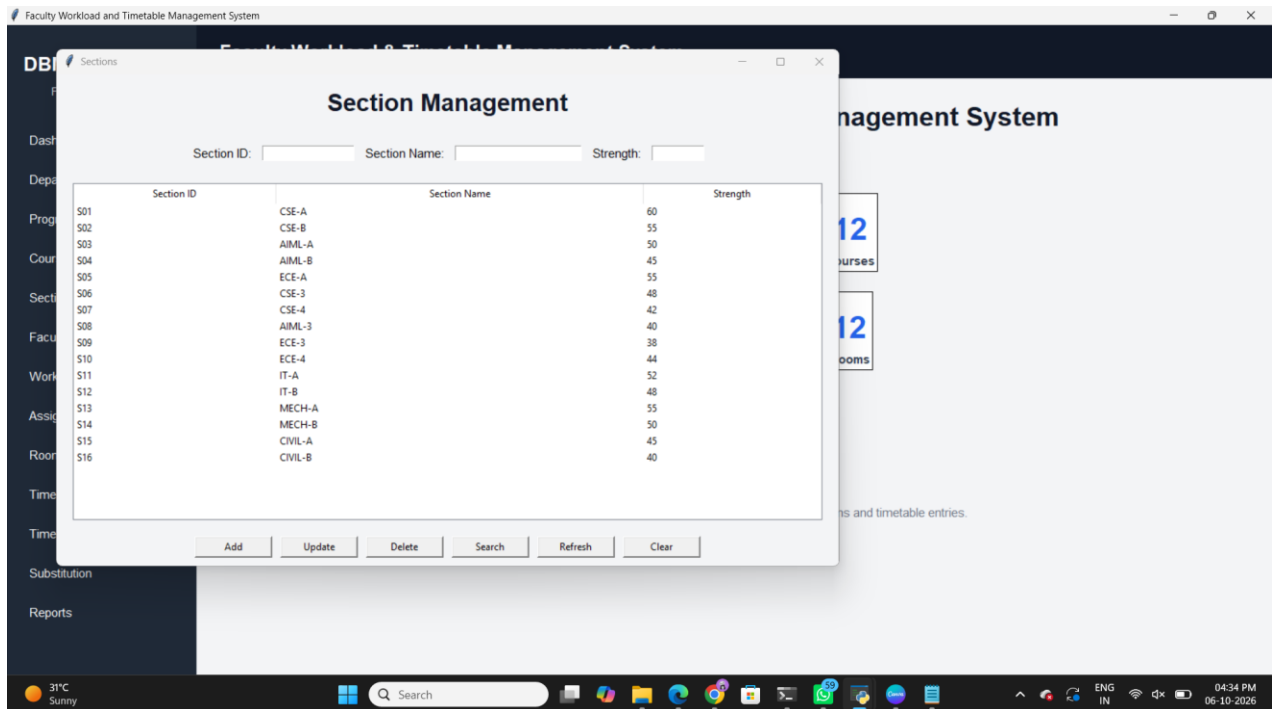
| Course ID | Course Name                            | Credits |
|-----------|----------------------------------------|---------|
| C01       | Database Management Systems            | 4       |
| C02       | Design and Analysis of Algorithms      | 4       |
| C03       | Computer Organization and Architecture | 3       |
| C04       | Machine Learning                       | 4       |
| C05       | Artificial Intelligence                | 4       |
| C06       | Digital Electronics                    | 3       |
| C07       | Operating Systems                      | 4       |
| C08       | Computer Networks                      | 4       |
| C09       | Software Engineering                   | 3       |
| C10       | Data Mining                            | 3       |
| C11       | Web Technologies                       | 3       |
| C12       | Cloud Computing                        | 4       |

Add Update Delete Search Refresh Clear

31°C Sunny 04:34 PM 06-10-2026

## 11.5 Sections Module

FIGURE 13: SECTIONS MODULE File



# 12. IMPLEMENTATION DETAILS

## 12.1 Technology Stack

- Frontend: Python 3 (Tkinter library) for desktop GUI layout, widget creation, modal forms, and event handling.
- Backend Database: MySQL 8.0 Server providing ACID-compliant relational data management.
- Connectivity Driver: mysql-connector-python facilitating standard SQL query execution, parameterized statements, and result fetch operations.
- Version Control: Git repository hosted on GitHub.

## 12.2 Database Connectivity

```
n
import mysql.connector
from mysql.connector import Error
```

```
def create_db_connection():
 try:
 connection = mysql.connector.connect(
 host="localhost",
 user="root",
 password="YOUR_MYSQL_PASSWORD",
 database="workload_timetable_db"
)
 if connection.is_connected():
 print("Successfully connected to MySQL database.")
 return connection
 except Error as e:
 print(f"Error connecting to MySQL: {e}")
 return None
```

### ## 12.3 Application Structure

The Python desktop frontend uses a structured modular layout:

- \* Main Window Container: Uses tkinter.ttk.Notebook to organize 11 dedicated functional modules cleanly into tabs.
- \* Form & Input Layer: Uses labeled Entry, Combobox, and Spinbox elements to collect parameters.
- \* Data Display Layer: Utilizes multi-column ttk.Treeview tables with scrollbars to display MySQL query results dynamically.

### ## 12.4 Key Code Snippets

Data Insertion Code Snippet:

```
```python

def add_department(dept_name_entry, tree):

    dept_name = dept_name_entry.get().strip()

    if not dept_name:
```

```
print("Validation Error: Department name cannot be empty.")
```

```
return
```

```
conn = create_db_connection()
```

```
if conn:
```

```
    cursor = conn.cursor()
```

```
    query = "INSERT INTO DEPARTMENT (dept_name) VALUES (%s)"
```

```
    try:
```

```
        cursor.execute(query, (dept_name,))
```

```
        conn.commit()
```

```
        print("Department added successfully.")
```

```
        dept_name_entry.delete(0, 'end')
```

```
        refresh_department_treeview(tree)
```

```
    except Error as e:
```

```
        print(f"Database Error: {e}")
```

```
    finally:
```

```
        cursor.close()
```

```
        conn.close()
```

Data Record Fetch & Display Snippet: `def refresh_department_treeview(tree):`

```
    for item in tree.get_children():
```

```
        tree.delete(item)
```

```
conn = create_db_connection()

if conn:

    cursor = conn.cursor()

    query = "SELECT dept_id, dept_name FROM DEPARTMENT ORDER BY dept_id
ASC"

    cursor.execute(query)

    rows = cursor.fetchall()

    for row in rows:

        tree.insert("", "end", values=row)

    cursor.close()

    conn.close()
```

Record Deletion Code Snippet: `def delete_department(selected_dept_id, tree):`

```
conn = create_db_connection()

if conn:

    cursor = conn.cursor()

    query = "DELETE FROM DEPARTMENT WHERE dept_id = %s"

    try:

        cursor.execute(query, (selected_dept_id,))

        conn.commit()

        print("Department deleted successfully.")

        refresh_department_treeview(tree)
```

except Error as e:

```
print(f"Delete Failed: {e}")
```

finally:

```
cursor.close()
```

```
conn.close()
```

13. TESTING

Test Case ID	Target Module	Input / Action
TC_01	Department	Insert new valid department name
TC_02	Department	Insert duplicate department name
TC_03	Faculty	Search faculty by partial string match
TC_04	Faculty	Select faculty row and trigger Update
TC_05	Faculty	Insert faculty with non-existent dept_id
TC_06	Section	Enter negative number for section strength
TC_07	Timetable	Insert timetable entry with valid IDs
TC_08	Substitution	Delete timetable entry referenced by substitution

Test Case ID	Target Module	Input / Action
TC_09	Common UI	Attempt submit with empty fields

14. CONCLUSION AND FUTURE ENHANCEMENTS

14.1 Conclusion

The Faculty Workload and Timetable Management System demonstrates a database-driven solution that simplifies academic scheduling and workload distribution tracking. By modeling entities in accordance with third normal form (3NF) principles, the system eliminates structural data redundancy while strictly enforcing foreign key relationships.

The MySQL backend ensures data consistency, while the Python Tkinter frontend delivers a simple desktop interface for administrative staff. The application provides complete CRUD functionality, search operations, and clear visibility across departments, programmes, courses, sections, faculty, workload norms, room allocations, time slots, timetable entries, and substitutions.

14.2 Future Enhancements

- Automated Conflict Detection: Implementing pre-insertion database triggers or frontend validation scripts that flag double-booked rooms or overlapping faculty schedules in real time.
- Algorithmic Timetable Generation: Integrating constraint-based scheduling algorithms (e.g., genetic algorithms) to automatically generate optimal timetables based on room capacity and workload rules.
- Role-Based Access Control (RBAC): Introducing user authentication mechanisms distinguishing administrative superusers, department heads, and viewing-only faculty accounts.
- Web-Based Deployment: Porting the desktop GUI to a cloud-based web framework (such as Django or Flask) to allow remote multi-tenant access.

- Export and Reporting Utility: Generating downloadable PDF schedules, Excel workload analytics, and official departmental timetable reports.

15. REFERENCES

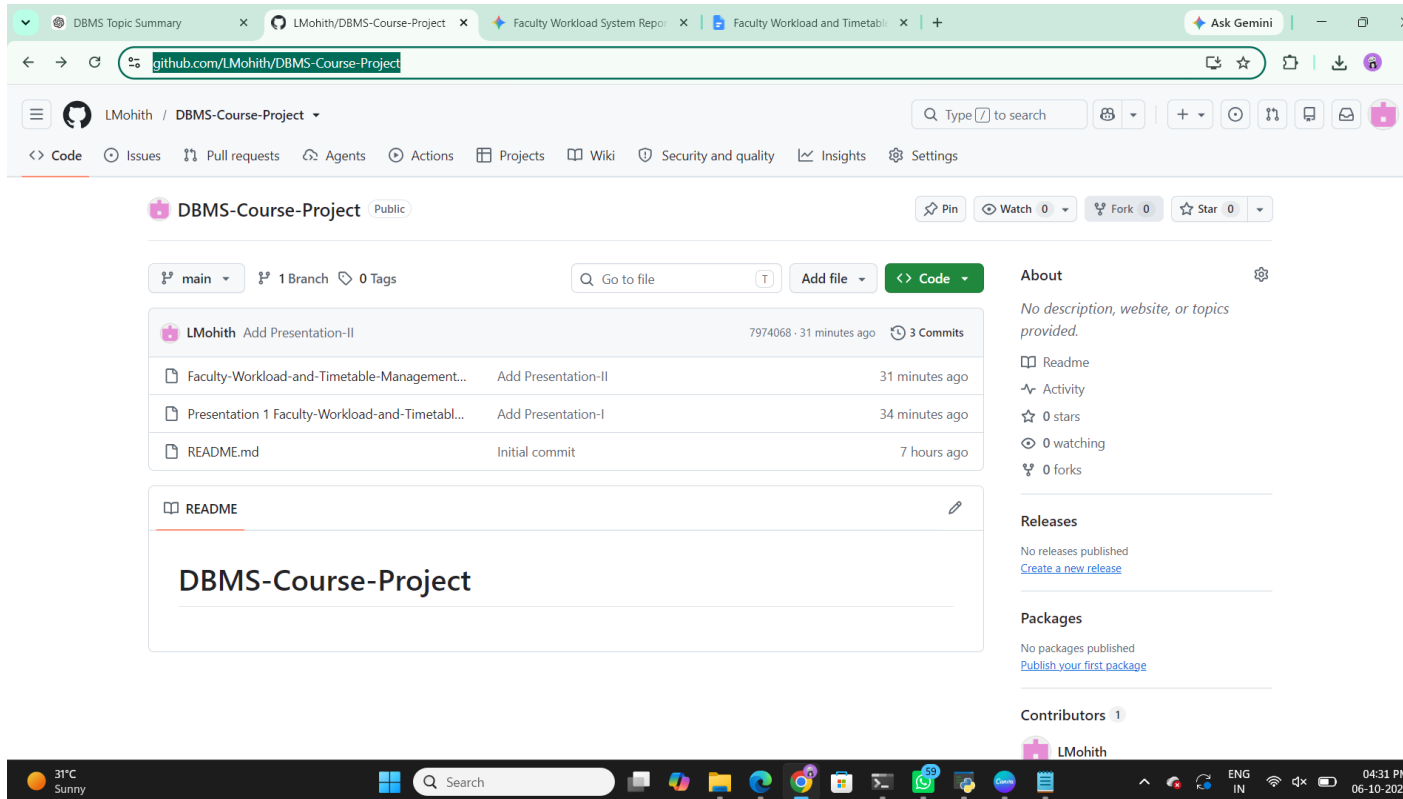
1. Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
3. MySQL Reference Manual: Official MySQL 8.0 Documentation. Available online: <https://dev.mysql.com/doc/>
4. Python Standard Library Documentation: Tkinter — Python interface to Tcl/Tk. Available online: <https://docs.python.org/3/library/tkinter.html>
5. MySQL Connector/Python Developer Guide: Official Oracle MySQL Connector documentation.

16. APPENDIX: GITHUB REPOSITORY

Repository Link:<https://github.com/LMohith/DBMS-Course-Project>

File

FIGURE 21: GITHUB REPOSITORY SCREENSHOT
File



The screenshot shows a web browser displaying the GitHub repository page for 'DBMS-Course-Project' by user 'LMohith'. The browser's address bar shows the URL 'github.com/LMohith/DBMS-Course-Project'. The repository page includes a header with the repository name, a search bar, and navigation links like 'Code', 'Issues', 'Pull requests', etc. Below the header, there's a section for the repository's main branch 'main', showing 1 branch and 0 tags. A table lists recent commits, including 'Add Presentation-II' and 'Add Presentation-I'. The 'README' section is visible, titled 'DBMS-Course-Project'. On the right side, there's a sidebar with 'About' (No description, website, or topics provided), 'Releases' (No releases published), 'Packages' (No packages published), and 'Contributors' (1 contributor: LMohith). The bottom of the image shows a Windows taskbar with various icons and a system tray displaying '31°C Sunny' and the date '06-10-2022'.